REST API

Redis Enterprise Software provides a REST API to help you automate common tasks.

Here, you'll find the details of the API and how to use it.

For more info, see:

- Supported request endpoints, organized by path
- Supported objects, both request and response
- Built-in roles and associated permissions
- Redis Enterprise Software REST API quick start with examples

Authentication

Authentication to the Redis Enterprise Software API occurs via Basic Auth. Provide your username and password as the basic auth credentials.

If the username and password is incorrect or missing, the request will fail with a `401 Unauthorized` status code.

Example request using cURL:

```
curl -u "demo@redislabs.com:password" \
    https://localhost:9443/v1/bdbs
```

For more examples, see the Redis Enterprise Software REST API quick start

Permissions

By default, the admin user is authorized for access to all endpoints. Use role-based access controls and role permissions to manage access.

If a user attempts to access an endpoint that is not allowed in their role, the request will fail with a `403 Forbidden` status code. For more details on which user roles can access certain endpoints, see Permissions.

Certificates

The Redis Enterprise Software REST API uses Self-signed certificates to ensure the product is secure. When you use the default self-signed certificates, the HTTPS requests will fail with `SSL certificate problem: self signed certificate` unless you turn off SSL certificate verification. The examples in this tutorial turn off SSL certificate verification.

Ports

All calls must be made over SSL to port 9443. For the API to work, port 9443 must be exposed to incoming traffic or mapped to a different port.

If you are using a Redis Enterprise Software Docker image, run the following command to start the Docker image with port 9443 exposed:

```
docker run -p 9443:9443 redislabs/redis
```

Versions

All API requests are versioned in order to minimize the impact of backwards-incompatible API changes and to coordinate between different versions operating in parallel.

Specify the version in the request URI, as shown in the following table:

Request path	Description
POST `/v1/bdbs`	A version 1 request for the `/bdbs` endpoint.
POST `/v2/bdbs`	A version 2 request for the `/bdbs` endpoint.

When an endpoint supports multiple versions, each version is documented on the corresponding endpoint. For example, the bdbs request page documents POST requests for version 1 and version 2.

Headers

Requests

Redis Enterprise REST API requests support the following HTTP headers:

Header	Supported/Required Values
Accept	`application/json`
Content-Length	Length (in bytes) of request message
Content-Type	`application/json` (required for PUT or POST requests)

If the client specifies an invalid header, the request will fail with a `400 Bad Request` status code.

Responses

Redis Enterprise REST API responses support the following HTTP headers:

Header	Supported/Required Values
Content-Type	`application/json`
Content-Length	Length (in bytes) of response message

JSON requests and responses

The Redis Enterprise Software REST API uses JavaScript Object Notation (JSON) for requests and responses. See the RFC 4627 technical specifications for additional information about JSON.

Some responses may have an empty body but indicate the response with standard HTTP codes.

Both requests and responses may include zero or more objects.

If the request is for a single entity, the response returns a single JSON object or none. If the request is for a list of entities, the response returns a JSON array with zero or more elements.

If you omit certain JSON object fields from a request, they may be assigned default values, which often indicate that these fields are not in use.

Response types and error codes

HTTP status codes indicate the result of an API request. This can be `200 OK` if the server accepted the request, or it can be one of many error codes.

The most common responses for a Redis Enterprise API request are:

Response	Condition/Required handling
200 OK	Success
400 Bad Request	The request failed, generally due to a typo or other mistake.
401 Unauthorized	The request failed because the authentication information was missing or incorrect.
403 Forbidden	The user cannot access the specified URI.
404 Not Found	The URI does not exist.
503 Service Unavailable	The node is not responding or is not a member of the cluster.
505 HTTP Version Not Supported	An unsupported `x-api-version` was used. See versions.

Some endpoints return different response codes. The request references for these endpoints document these special cases.

Updated: July 5, 2022